# JAVA PROGRAMMING PRACTICES

## Valid Parentheses Using Stack

### Basic Information

| | |
|---|---|
| Week | Week 14 |
| Source file | ValidParentheses.java |
| Concepts | Collection classes, Deque stack operations, conditions. |

## Objective / Aim

To implement valid parentheses using stack using the concepts listed in the R23 Java Programming Practices syllabus.

## System Requirements

JDK (Java Development Kit) and a text editor or Java IDE.

## Algorithm / Logic Description

1. Push opening brackets onto a Deque.
2. Match each closing bracket with the top of the stack.
3. Display whether the brackets are balanced.

## UML / Program Structure

```
ValidParentheses
+ isValid(text) : boolean
Uses Deque as a stack
```

## Source Code

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;
public class ValidParentheses
{
    static boolean isValid(String text)
    {
        Deque<Character> stack = new ArrayDeque<Character>();
        for (int i = 0; i < text.length(); i++)
        {
            char ch = text.charAt(i);
            if (ch == '(' || ch == '[' || ch == '{') stack.push(ch);
            else if (ch == ')' || ch == ']' || ch == '}')
            {
                if (stack.isEmpty()) return false;
                char open = stack.pop();
                if ((ch == ')' && open != '(') || (ch == ']' && open != '[')
                || (ch == '}' && open != '{')) return false;
            }
        }
        return stack.isEmpty();
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter bracket expression: ");
        String text = sc.nextLine();
        System.out.println(isValid(text) ? "Valid Parentheses" : "Invalid Parentheses");
        sc.close();
    }
}
```

## Compilation and Execution

```
javac ValidParentheses.java
java ValidParentheses
```

## Output Screen

```
Enter bracket expression: Valid Parentheses
```

## Result

The program demonstrates the listed syllabus concepts. The sample output is based on a compiled execution.